

Lumap 软件整体架构

macOS 与 iOS 启发式学习系统的当前实现、信任边界与演进设计

Celestial Frontier

版本 1.0 2026 年 9 月 30 日

摘要

本文给出 Lumap 的软件整体架构，依据仓库中 macOS 0.1.0 与 iOS 0.2.0 的实际代码建立当前实现基线，并将个性化推荐、真实空间计算、跨设备同步等后续能力标为目标架构。文档采用可复现的 XeLaTeX 排版，所有架构图均由 TikZ 生成。当前应用是本地优先的 SwiftUI/SwiftData 产品：两端共享模型、业务状态、供应商客户端和语音服务；API 密钥保存在各自 Keychain；17 种学习方法通过统一完成合同写入活动证据；Narrated Deck 可调用户配置的远程模型并在失败时明确回退到本地确定性课件；Kokoro 与 sherpa-onnx 在设备上生成旁白；空间学习当前保留可交互的 2D fallback。目标 Recommendation Plane 仅在本文中定义接口与约束，尚未部署为在线策略服务。

关键词：启发式学习；SwiftUI；SwiftData；个性化推荐；离线优先；Kokoro；ARKit；信任边界

阅读约定。图中的实线框和实线箭头表示当前仓库已存在的组件或运行路径；棕色虚线框和虚线箭头表示目标组件或尚待实现的链路。架构事实截止到本文版本日期。

目录

1	范围、目标与事实基线	3
1.1	设计目标	3
1.2	当前工程基线	3
1.3	非目标与明确边界	4
2	当前运行架构	4
2.1	表现层与状态层	5
2.2	持久化模型	5
2.3	17 种学习方法的组件合同	5
3	端到端数据流与信任边界	5
3.1	一次学习会话的数据路径	5
3.2	数据分类与最小披露	7
3.3	威胁面与控制	7

4	Narrated Deck 与本地语音管线	7
4.1	远程生成和确定性回退	7
4.2	Kokoro 音频路径	8
5	空间学习与跨端组件	8
5.1	当前 capability 模型	8
5.2	目标 renderer 合同	10
6	目标 Recommendation Plane	10
6.1	职责边界	10
6.2	状态、候选与决策合同	11
6.3	训练、评估与上线闸门	11
7	部署拓扑、故障隔离与回退	11
7.1	故障矩阵	11
7.2	可观测性与服务目标	11
8	接口与数据结构	13
8.1	当前关键接口	13
8.2	目标请求信封	13
8.3	事件顺序与幂等性	14
9	演进架构与交付顺序	14
9.1	从 MVP 到可审计客户端	14
9.2	从确定性推荐到 RL shadow	14
9.3	从 2D 空间演示到真实 renderer	14
9.4	优先级与验收门槛	14
10	验证策略	15
10.1	构建与单元测试	15
10.2	架构级故障注入	15
10.3	可访问性与跨端验收	15
11	结论	15
A	当前代码映射	16
B	图例与编译说明	16

1 范围、目标与事实基线

1.1 设计目标

Lumap 面向全年龄、多学科和多种表达方式的学习者。系统的工程目标不是把所有学习都交给一个远程模型，而是建立一个可以验证、可以降级并且允许学习者纠正的闭环：输入学习意图，选择学习方法，产生可保存的学习证据，选择是否进行理论或实践检测，再以这些结果调整后续建议。

架构遵循以下原则：

- 1. 本地可运行：创建目标、17 种活动、进度、奖励和 Persona 不依赖网络。
- 2. 用户意图优先：用户明确输入的学习主题高于画像推断和推荐结果。
- 3. 证据分层：接触、练习、检测与长期保持分别记录，不把一次点击当作掌握。
- 4. 能力可替换：远程模型、语音引擎和空间 renderer 均通过边界接口进入，不直接拥有数据库写权限。
- 5. 降级可见：网络、语音包或 AR 能力缺失时展示真实状态，不把样例输出冒充在线结果。
- 6. 当前与目标分离：运行中的确定性推荐与未来 Recommendation Plane 使用不同状态标签和事件合同。

1.2 当前工程基线

表 1 当前代码基线

维度	当前值	说明
客户端	macOS 14+, iOS 18+	原生 SwiftUI；分别使用 com.local.lumap 与 com.local.lumap.ios。
语言与并发	Swift 6	工程启用主 actor 默认隔离；语音引擎池使用独立 actor 串行化推理。
本地存储	SwiftData	当前每个应用容器包含一个本地 Lumap store；尚未实现多档案物理隔离和版本迁移计划。
密钥	Keychain	服务名为 com.local.lumap.provider；密钥不写入 SwiftData、日志或仓库。
学习活动	17 种 renderer	各方法有独立交互案例，完成后写 ActivityRecord。
远程生成	可选	支持 OpenAI Chat、Responses 与 Anthropic Messages 形状；当前只有 Narrated Deck 接入生成。
本地语音	可选真实推理	sherpa-onnx 1.13.8 与 Kokoro multilingual v1.1；语音包缺失时进入 silent preview。
空间学习	跨端原型	iOS 检测 ARKit 世界跟踪能力；当前学习 renderer 始终提供 Camera Off 的 2D 交互效果。
推荐策略	确定性批次	三批本地候选可刷新；LumaPath-RL 尚未训练或部署。

1.3 非目标与明确边界

当前版本没有跨设备账户同步、云端用户数据库、真实社交平台爬取、长期系统级屏幕监测、在线 RL 策略服务或 visionOS target。iOS 的 ARKit capability 检查不等于已有真实相机 AR renderer；公开主页输入当前仅生成本地确定性兴趣预览。未来 Apple 空间设备只能通过公开 SDK 和显式 target 接入，本文不以未发布硬件作为依赖。

2 当前运行架构

图 1 描述已经存在的主要代码路径。两端共享 LumapStore、SwiftData 模型、LumapAIClient、Narrated Deck 合同、Keychain 包装器和 Kokoro 包管理器；平台层分别提供窗口、导航、ARKit capability 与音频会话适配。

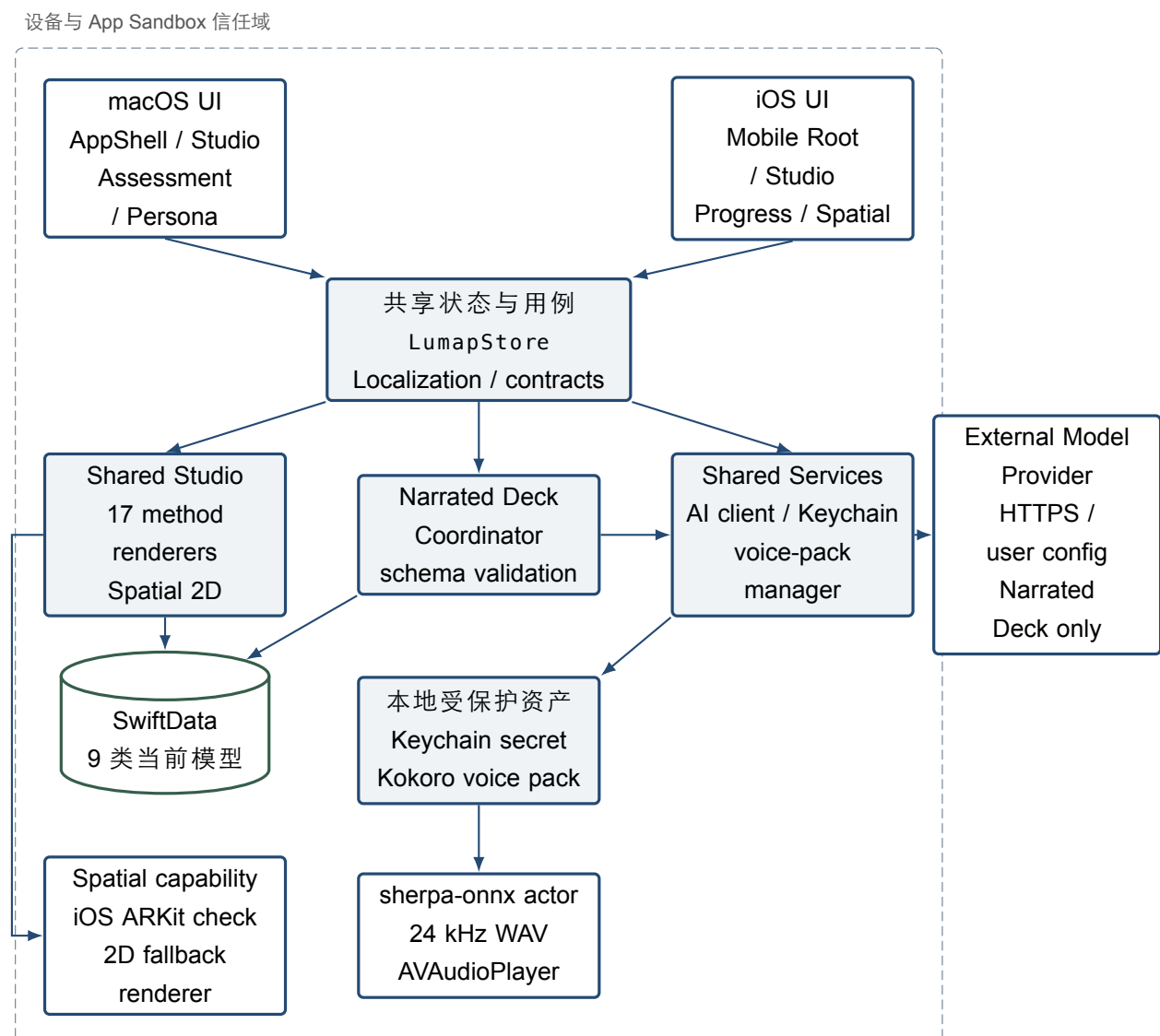


图 1 当前运行架构。远程模型网关位于设备信任域之外，且只在用户完成供应商配置后参与 Narrated Deck 生成。

2.1 表现层与状态层

macOS 入口为 LumapApp，主窗口使用 AppShellView。桌面 Persona 由 MenuBarExtra 与 PersonaPanelController 提供。iOS 入口为 LumapiOSApp，采用移动端独立视图。两端共享领域模型和主要服务文件。LumapStore 是当前主线程状态编排器，负责创建目标、推荐批次、活动完成、检测、奖励与学习模式状态，并通过 SwiftData 的 ModelContext 持久化。

这一结构适合 Hackathon MVP，但 LumapStore 同时承担 UI 状态、用例编排和持久化写入。目标版本应将可测试用例与 Repository 移入 actor，UI 仅消费不可变快照，见第 9 节。

2.2 持久化模型

当前 SwiftData schema 包含九类模型。档案与来源层包括 LearnerProfile、SourceRecord 和 InterestEvidence；学习执行层包括 LearningGoal、ActivityRecord 与 AssessmentRecord；其余是 RewardEntry、MaterialRecord 与 AppHealthRecord。核心关联使用稳定 UUID。活动和检测分别用 goalID 关联目标；奖励使用 eventKey 保证同一事件不重复入账。

对目标 g ，MVP 进度由首次出现的不同活动方法、理论检测与实践检测离散推进。可将当前逻辑写为

$$p_g = \min\left(1, \frac{|M_g| + I_g^{\text{theory}} + I_g^{\text{practical}}}{5}\right), \quad (1)$$

其中 M_g 是目标 g 下已完成且按方法去重的集合， I 是是否存在相应完成证据的指示量。式 (1) 只是演示进度投影，不等同于概念掌握概率。

2.3 17 种学习方法的组件合同

当前方法枚举包含引导讲解、示例拆解、苏格拉底对话、类比、概念图、分支剧情、闪卡回忆、反向讲解、互动模拟、Spatial AR、刻意练习、反思、误区诊断、好奇分支、反事实实验、迁移挑战和 Narrated Deck。各 renderer 保留自己的输入状态和交互语义，但最终通过共同合同提交：

$$\text{complete}(g, m, a, h, k) \longrightarrow (\text{ActivityRecord}, \text{RewardEntry?}, p'_g), \quad (2)$$

其中 g 为目标， m 为方法， a 为学习产出文本， h 为帮助程度， k 为幂等事件键。当前写入由应用层同步协调；目标 Repository 应把活动、奖励与待投影事件置于同一逻辑事务。

3 端到端数据流与信任边界

3.1 一次学习会话的数据路径

图 2 从输入到证据展示给出完整路径。网络生成只是一条可选分支；本地创建目标、活动交互、保存证据和查看进度不需要远程服务。

远程分支只发送主题、可选资料摘录、教学语言和精简学习者上下文；API secret 仅进入 HTTPS 认证头。

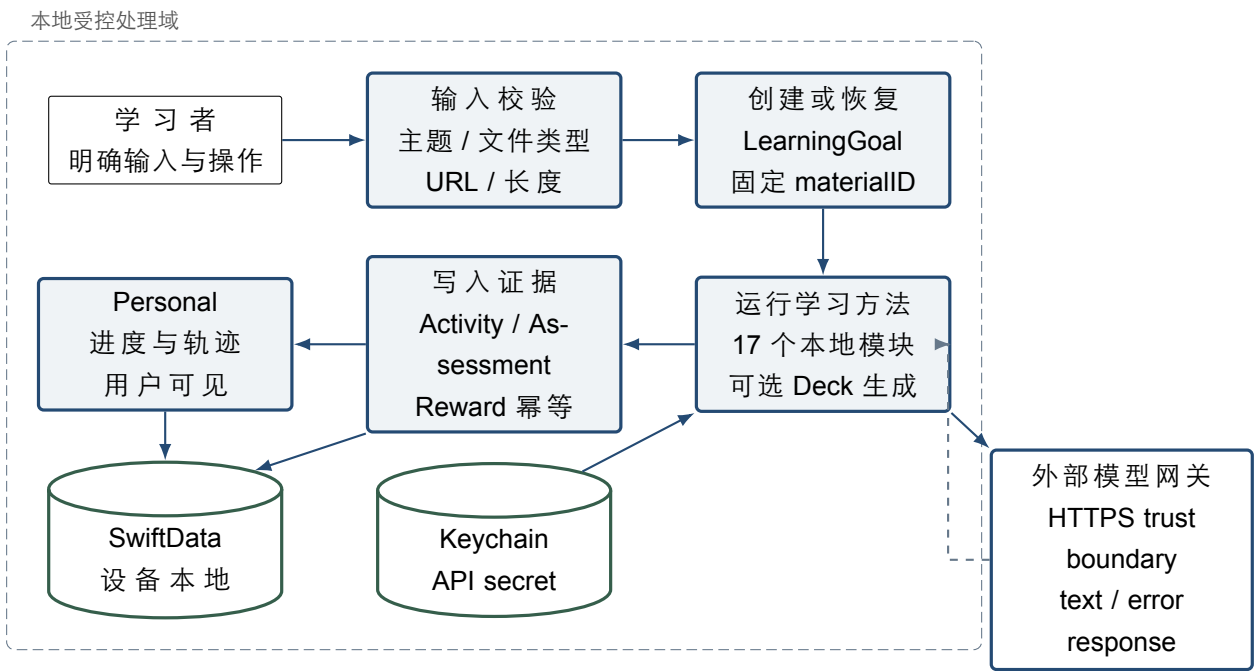


图 2 端到端学习数据流。密钥仅从 Keychain 进入认证头，不写入生成 prompt 或学习记录。

表 2 数据分类、位置与出域条件

数据	默认位置	可否发送到远程模型	控制
学习目标与语言	SwiftData / 内存	Narrated Deck 配置后发送	用户主动开始生成
资料摘录	SwiftData 中的限额 excerpt	仅该课件请求所绑定摘录	UI 显示来源标签；失败可本地回退
年龄段与背景	LearnerProfile	当前可形成精简 learnerContext	应继续限制字段并提供关闭选项
活动与检测历史	SwiftData	当前不发送	目标推荐只使用允许列表特征
API 密钥	Keychain	仅作为 HTTPS 认证头	不进入日志、导出、prompt 或 Git
Kokoro 权重与 WAV	Application Support / 临时目录	不发送	语音在设备上合成，临时 WAV 播放后删除
AR 相机帧	当前不采集	当前不发送	2D fallback 明示 Camera Off

3.2 数据分类与最小披露

远程生成请求的允许内容集合记为 A ，用户当前授权集合记为 C_t ，实际出域字段必须满足

$$X_{\text{remote},t} \subseteq A \cap C_t, \quad \text{secret} \notin X_{\text{remote},t}. \quad (3)$$

授权撤回或目标 revision 改变后，尚未落库的响应必须失效。当前 MVP 已隔离 Keychain 与 SwiftData，但完整 consent revision 和请求注册表属于目标架构。

3.3 威胁面与控制

表 3 主要威胁与控制

威胁	影响	控制与剩余工作
恶意资料中的指令	模型把资料当系统指令	Deck adapter 明确要求把 source material 当数据，响应仅接受固定 schema；仍需在生产环境加入内容限额与审计。
伪造或越界模型响应	UI 崩溃、错误 Quiz	校验 schemaVersion、3-8 页、连续页码、视觉枚举、强度范围、Quiz 选项与正确答案。
凭据泄漏	账户费用和数据风险	Keychain 存储，设置快照不含密钥，请求只用认证头；发布前补充日志脱敏测试和重定向策略测试。
重复完成或重放	奖励重复、进度失真	RewardEntry.eventKey 与活动查询去重；目标事件层应加入 sequence 和 requestID。
损坏语音包	崩溃或加载任意文件	只复制白名单、拒绝符号链接和特殊文件、验证模型与 voices 文件大小；归档 SHA-256 由安装脚本验证。
迟到异步结果	覆盖新主题或撤回后的数据	语音已支持取消回调；远程生成目标需进一步携带 profile/goal/input revision 并在提交前核对。

4 Narrated Deck 与本地语音管线

4.1 远程生成和确定性回退

Deck 请求使用版本化的 NarratedDeckGenerationRequest，包含主题、可选资料摘录及来源标签、语言、精简学习者上下文和最大页数。远程响应必须解码为 NarratedDeckGenerationResponse，每页包含稳定 ID、标题、2-4 个要点、讲稿、受支持视觉类型和可选 Quiz。图 3 给出当前顺序。

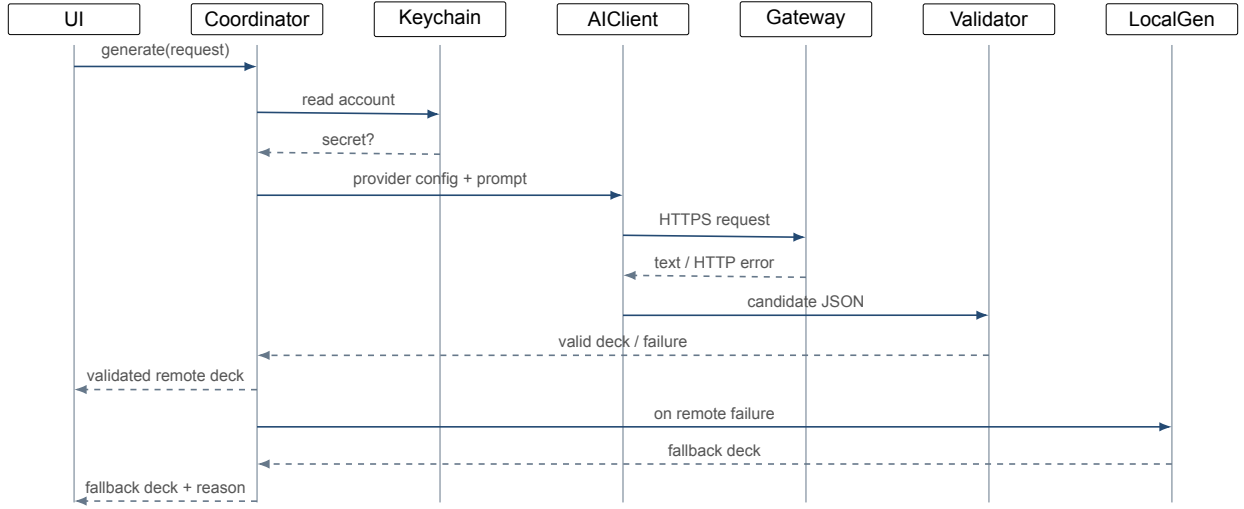


图 3 Narrated Deck 当前生成时序。回退结果带有来源说明，不能冒充实时远程生成。

远程结果的接受条件可以形式化为谓词

$$\text{ValidDeck}(d) = [d.v = v_0] \wedge [3 \leq |d.slides| \leq 8] \quad (4)$$

$$\wedge \bigwedge_{i=1}^{|d.slides|} [d_i.index = i] \wedge [d_i.visual \in \mathcal{V}] \wedge [0 \leq d_i.intensity \leq 1] \quad (5)$$

$$\wedge \bigwedge_{q \in d.quiz} [q.correctID \in q.optionIDs], \quad (6)$$

其中 $\mathcal{V}$ 是编译期允许的视觉枚举。若谓词为假、HTTP 失败、密钥缺失或供应商不可用，Coordinator 使用固定模板的 `LocalNarratedDeckGenerator` 并保留失败原因供 UI 展示。

4.2 Kokoro 音频路径

语音实现不调用系统 TTS。SherpaKokoroEnginePool 是进程级 actor，缓存一个重量级 sherpa-onnx 引擎并串行化推理；生成回调读取线程安全取消标记，从而允许换页和停止操作中中断合成。有效音频保存为 24 kHz WAV，经 AVAudioPlayer 播放并在停止、完成或失败后清理临时文件。iOS 另设置 spoken-audio 音频会话并处理系统中断。

语音包放在 App Sandbox 的 Application Support 下。macOS 可由固定脚本预装；iOS Settings 允许用户选择已经解压的目录，导入器验证必需文件、拒绝符号链接并先写 staging，再原子替换旧包。语音包缺失或无效时，VoicePackRequiredNarrationProvider 只推进 silent-preview 时间线、字幕和 Quiz，不产生声音，也不回退到 AVSpeechSynthesizer。

5 空间学习与跨端组件

5.1 当前 capability 模型

共享空间协议暴露 availability、scenario、prepare 与 reset；代码类型为 SpatialLearningProviding。iOS 的 MobileARKitProvider 在真机检查 ARKit 世界跟踪支持，Simulator 返回 preview-only。macOS 和 iOS 界面都保留可点击的 2D 空间实验。当前 renderer 明确显示 Camera Off / 2D Fallback，因此视觉效果是演示空间交互，不是对真实环境的测量。

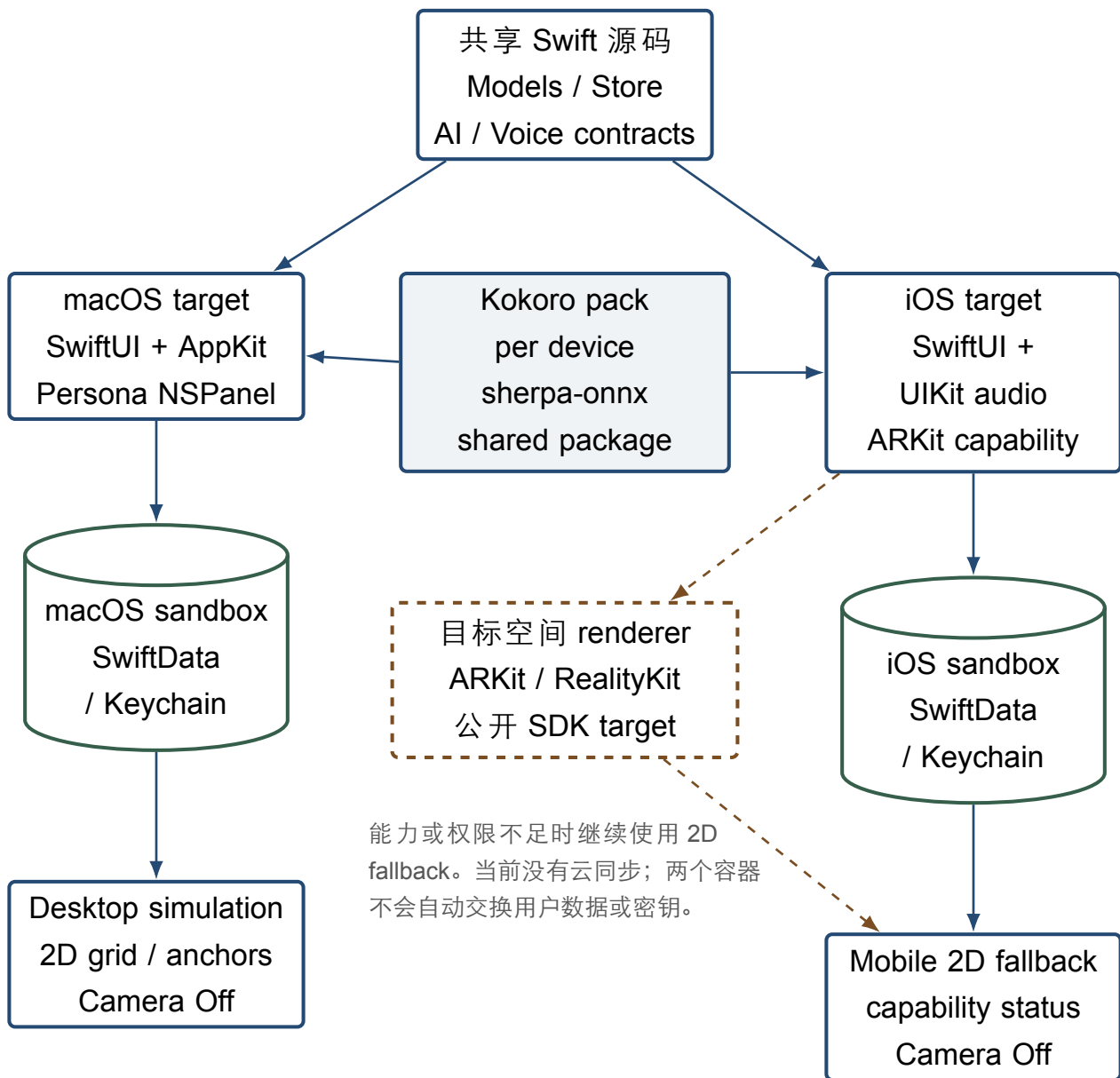


图 4 跨端组件、独立容器与空间能力降级。棕色虚线组件是目标 renderer。

5.2 目标 renderer 合同

目标空间层需要把设备能力、权限和运行状态与教学活动分开。接口输入采用版本化 session request，输出带坐标系版本的 Observation；请求类型为 SpatialSessionRequest。状态机为

$$\text{checking} \rightarrow \begin{cases} \text{ready} \rightarrow \text{tracking} \rightarrow \text{paused/stopped,} & \text{能力与权限可用,} \\ \text{previewOnly,} & \text{Simulator、拒绝权限或跟踪失败,} \\ \text{unavailable,} & \text{无任何 renderer.} \end{cases} \quad (7)$$

二维屏幕坐标、相机像素坐标与世界坐标必须使用不同的类型和版本；renderer 只产生 observation，不直接修改掌握度或奖励。

6 目标 Recommendation Plane

6.1 职责边界

Recommendation Plane 的任务是从允许的候选活动中返回下一步决定及其解释，而不是直接创建数据库记录或绕过安全条件。当前产品只运行确定性推荐批次；本节全部标为**目标架构**。推荐服务应支持两类决定：主页发现内容与学习会话内的 concept-method-dose 组合。

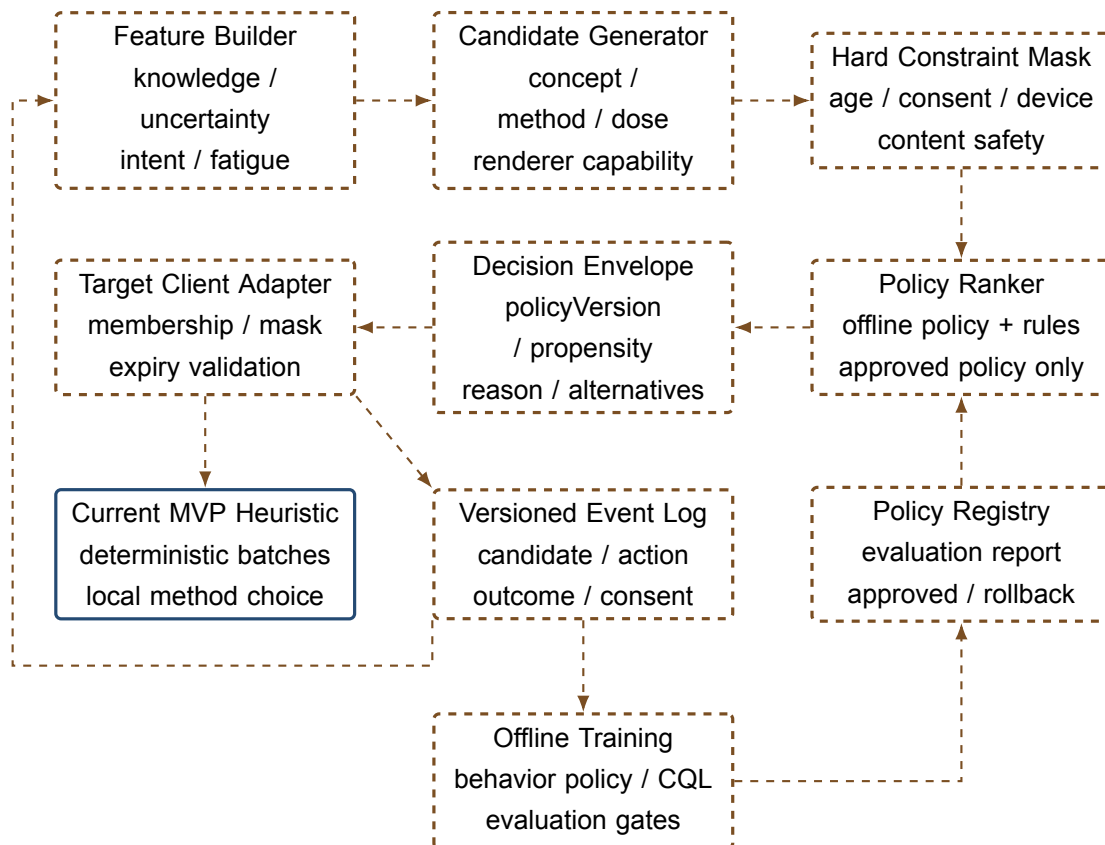


图 5 目标 Recommendation Plane。棕色虚线组件尚未部署；蓝色实线框是当前确定性 heuristic，也是目标 Adapter 的失败回退。

6.2 状态、候选与决策合同

学习者状态只从已授权的允许列表特征构造：

$$s_t = (k_t, u_t, e_t, m_t, i_t, f_t, q_t, g_t, c_t), \quad (8)$$

分别表示知识估计、不确定性、参与度、方法偏好、明确意图、疲劳、证据质量、当前目标和设备能力。候选动作采用层级结构

$$a_t = (\text{intent}, \text{concept}, \text{method}, \text{dose}, \text{content}). \quad (9)$$

在策略打分之前，硬约束 $H(s_t, a)$ 必须形成候选掩码

$$\mathcal{A}_t^{\text{safe}} = \{a \in \mathcal{A}_t \mid H(s_t, a) = 1\}, \quad a_t^* = \arg \max_{a \in \mathcal{A}_t^{\text{safe}}} Q_\theta(s_t, a). \quad (10)$$

年龄适宜性、授权、设备能力和内容安全属于硬约束，不能被高学习奖励抵消。Decision Envelope 至少包含 decisionID、动作、候选集哈希、每层 propensity、结构化理由、policyVersion、consentRevision 和失效时间。

6.3 训练、评估与上线闸门

目标离线训练可采用 Conservative Q-Learning。训练数据先经过硬约束过滤，记为 D_{safe} 。一个简化目标为

$$\mathcal{L}_{\text{TD}}(\theta) = \mathbb{E}_{(s,a,r,s') \sim D_{\text{safe}}} \left[\left(Q_\theta(s, a) - r - \gamma \max_{a' \in \mathcal{A}^{\text{safe}}(s')} Q_{\bar{\theta}}(s', a') \right)^2 \right], \quad (11)$$

$$\mathcal{L}_{\text{CQL}}(\theta) = \mathcal{L}_{\text{TD}}(\theta) + \alpha \mathbb{E}_{s \sim D_{\text{safe}}} \left[\log \sum_{a \in \mathcal{A}^{\text{safe}}(s)} e^{Q_\theta(s,a)} - \mathbb{E}_{a \sim D_{\text{safe}}(\cdot|s)} Q_\theta(s, a) \right]. \quad (12)$$

总体奖励可以组合即时掌握增益、延迟保持、迁移、自主性和目标进展，同时扣除挫败、过载、无效重复和提示依赖：

$$r_t = \sum_i w_i r_{i,t}, \quad J_{C_j}(\pi) = \mathbb{E}_\pi \left[\sum_{t=0}^T \gamma^t c_{j,t} \right] \leq d_j. \quad (13)$$

隐私、年龄、授权和内容安全由式 (10) 的 hard mask 处理。CMDP 只在合法动作集合内约束疲劳、过载和打扰等累积软风险，不能放宽任何硬约束。上线前需要离线策略评估、年龄和语言分层、覆盖率、最差分位表现、fallback 率、人工红队与可回滚 policy registry。无合法候选、超时、签名或版本不匹配时必须走版本化 heuristic fallback。

7 部署拓扑、故障隔离与回退

7.1 故障矩阵

7.2 可观测性与服务目标

当前应用应记录不含个人原文和密钥的本地诊断事件：组件、状态转移、延迟桶、错误类别、fallback 原因与版本。目标 Recommendation Plane 另记录完整候选集合、hard mask、选中概率、policy version 和延迟结果，以支持离线评估。建议的服务目标为：本地目标创建和活动保存 p95 小于 100

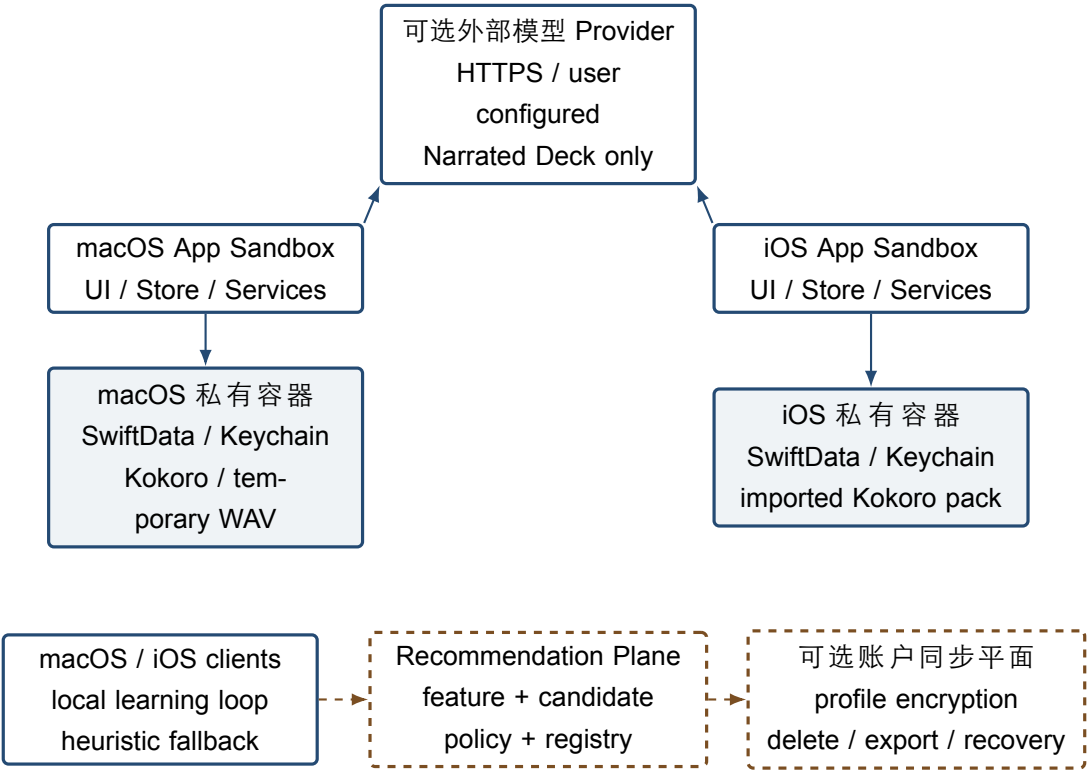


图 6 部署和故障域。上半为相互独立的当前设备容器；下排虚线为目标服务，不能成为本地学习闭环的单元故障。

表 4 故障检测、用户行为与数据保证

故障	用户可见回退	数据保证
SwiftData 打开失败	恢复页，允许重试，不进入空应用	不删除、不自动创建替代空库
Provider 配置不完整	显示未配置，生成本地 Deck	密钥不离开 Keychain
HTTP、网关或合同失败	标注 local fallback 及原因	无效响应不写活动证据
Kokoro 包缺失/损坏	silent preview、字幕和 Quiz 保留	不调用系统 TTS，不加载未验证文件
语音合成取消	停止或换页立即取消	临时 WAV 清理，旧任务不得恢复播放
iOS 音频中断	暂停，按系统恢复标志决定是否续播	playback state 与真实播放器一致
AR 不支持/拒绝权限	2D spatial preview	不产生真实世界跟踪结论
目标推荐服务超时	版本化 heuristic fallback	非法或过期 decision 被客户端拒绝

ms；本地 UI 交互保持 60 fps；推荐决定服务端 p95 小于 300 ms；任意远程失败在一次请求预算内显式回退。

系统可用性不是各组件可用性的简单乘积，因为本地 fallback 提供替代路径。若远程路径成功概率为 A_r 、本地回退成功概率为 A_f ，且两者失败近似独立，则 Deck 生成的功能可用性为

$$A_{\text{deck}} = A_r + (1 - A_r)A_f. \quad (14)$$

该式用于说明降级架构价值，不替代真实故障相关性测试。

8 接口与数据结构

8.1 当前关键接口

表 5 主要接口及所有权

接口/类型	所有者	约束
LearningMethod	共享 Models	固定 17 个枚举值；renderer 不应只换标题复用同一交互。
LumapStore	当前应用层	主 actor；承担用例与 UI 状态，后续拆分。
LumapAIClient	共享 Services	解析 endpoint，支持三种 wire style，45 秒超时，响应提取为文本。
Deck generation request	Deck service	NarratedDeckGenerationRequest 固定 schema version、topic、excerpt、language、context 与 3-8 页上限。
NarrationProviding	Voice boundary	availability、playbackState、progress、mute 和播放控制。
Spatial provider contract	Spatial boundary	SpatialLearningProviding 暴露 capability、scenario、prepare、reset；当前不携带真实相机帧。
Voice-pack manager	Local asset boundary	KokoroVoicePackManager 验证 allowlist、文件类型与关键大小，导入到固定目录。

8.2 目标请求信封

跨 actor 或跨网络的异步用例应携带版本化信封：

$$E = (\text{requestID}, \text{profileID}, \text{profileRevision}, \text{goalID}, \text{goalRevision}, \text{consentRevision}, \text{inputRevision}, \text{createdAt}, \text{expiresAt}). \quad (15)$$

结果提交条件为

$$\text{Commit}(R, E, t) \iff \text{ValidSchema}(R) \wedge t < E.\text{expiresAt} \wedge \text{Current}(E) \wedge \neg \text{Cancelled}(E.\text{requestID}). \quad (16)$$

这避免远程课件、推荐或未来 OCR 的迟到结果覆盖学习者已经修改的目标、档案或授权。

8.3 事件顺序与幂等性

目标事件格式 `ActivityEvent(instanceID, sequence, action, payloadHash)` 要求同一实例的 `sequence` 单调递增。Repository 对幂等键 k 的写入满足

$$\text{apply}(k, e) = \begin{cases} \text{commit}(e), & k \notin K, \\ \text{no-op}, & k \in K, \end{cases} \quad (17)$$

其中 K 是已提交事件键集合。重复网络重试、UI 双击和恢复重放因此不会重复奖励或推进进度。

9 演进架构与交付顺序

9.1 从 MVP 到可审计客户端

第一阶段保持界面与 17 个 `renderer` 不变，将 `LumapStore` 中的持久化写入迁至 `@ModelActorLumapRepository`，引入 `Sendable DTO`、事务幂等键和显式 `schema migration`。每个档案使用独立 `store`、附件、索引和 `Keychain account`；备份采用版本化逻辑快照，`Keychain` 密钥永不导出。

第二阶段为资料导入建立文档、片段与 `citation` 模型，加入文件生命周期和删除级联；为异步请求加入第 8 节定义的 `revision` 信封。远程模型调用继续是可选增强，推荐、检测与 `Persona` 在各自 `adapter` 通过评审前不共享一个无限权限 `prompt`。

9.2 从确定性推荐到 RL shadow

`Recommendation Plane` 的交付顺序应为：先记录可重放事件与候选集，再离线生成 `decision`，随后以 `shadow` 模式比较 `heuristic` 与 `policy`，最终只对满足闸门的流量小比例启用。每次上线绑定不可变 `policy version`，并保留一键回退到 `heuristic`。没有 `propensity`、候选集或 `consent revision` 的历史事件不得用于无偏策略评估。

9.3 从 2D 空间演示到真实 `renderer`

先在 iOS 真机建立相机会话、权限、跟踪质量和坐标类型测试，再通过共享空间协议的扩展合同注入 `RealityKit renderer`。跟踪丢失或设备不支持时回到现有 2D 活动；只有真实 `session` 产生且带坐标系版本的 `observation` 才能标记为 `camera-assisted evidence`。新的 `visionOS` 或未来设备 `target` 必须等待公开 SDK，并复用同一证据语义。

9.4 优先级与验收门槛

表 6 建议交付顺序

阶段	主要工作	退出条件
H0 基线	当前 macOS/iOS、17 模块、双类检测、Deck、Kokoro、2D spatial	两端构建；关键单元测试通过；无密钥进入仓库；fallback 状态可见。

阶段	主要工作	退出条件
H1 数据可靠性	VersionedSchema、Repository actor、逻辑导出/恢复、资料引用生命周期	迁移、磁盘满、崩溃注入、重复事件和删除级联 fixture 全部通过。
H2 个性化观测	候选集、曝光、结果、consent revision、方法效果比较	事件可重放；用户可查看和删除来源；缺失数据不被当作负反馈。
H3 RL shadow	离线训练、OPE、policy registry、客户端验证器	分层安全门槛通过；shadow 无写权限；随时回退 heuristic。
H4 空间与同步	真实 AR renderer、可选账户同步和设备恢复	真机权限/中断测试、坐标一致性、端到端删除与恢复通过。

10 验证策略

10.1 构建与单元测试

当前工程应至少执行 macOS scheme 的完整测试和 iOS Simulator 通用构建。测试覆盖 endpoint 解析、OpenAI Chat/Responses/Anthropic 文本提取、Deck schema 拒绝、Kokoro 包验证、奖励幂等、进度推进、Keychain 无明文序列化和数据库打开失败路径。外部网关连通性属于独立集成测试，不能使本地单元测试依赖真实密钥或计费账户。

10.2 架构级故障注入

建议对以下断点进行自动或手动故障注入：HTTP 400/429/500、无响应和非 JSON；模型返回越界页数和错误 Quiz ID；语音包缺文件、大小不符和符号链接；合成中取消与音频中断；SwiftData 打开失败和保存失败；AR Simulator、拒绝权限和跟踪中断；推荐 decision 过期、不在候选集、违反 hard mask 或来自未知 policy version。

10.3 可访问性与跨端验收

两端验证英文默认与中英文切换、动态字号、Reduce Motion、键盘或 VoiceOver 路径、窄屏无横向溢出。Narrated Deck 必须在无音频时保留字幕和 Quiz；空间实验在无摄像头时保留完整的 2D 操作；Persona 提示不能阻断学习任务或自动启动外部观察。

11 结论

Lumap 当前已经形成一个本地优先、跨 macOS 与 iOS 的可演示闭环。其可信度来自清晰的能力边界：SwiftData 记录学习证据，Keychain 隔离供应商密钥，17 种 renderer 保持不同交互语义，Narrated Deck 的远程输出经过 schema 校验并具有确定性本地回退，Kokoro 在设备上生成拟人旁白，空间活动在真实 AR 尚未接入时明确维持 2D fallback。

下一阶段的核心不是扩大一个通用模型的权限，而是把持久化、事件、授权 revision 和候选集合

做成可审计基础，再以 shadow 方式引入 Recommendation Plane。本文的实线/虚线区分应在每次版本更新时同步维护，使架构图继续反映可运行事实，而不是只描述愿景。

A 当前代码映射

路径	架构职责
Lumap/App/LumapApp.swift	macOS scene、数据库 bootstrap、恢复界面与独立本地 schema。
LumapiOS/App/LumapiOSApp.swift	iOS scene、移动端数据库打开与恢复入口。
Lumap/Models/LumapModels.swift	九类 SwiftData 模型、17 种 LearningMethod 与核心枚举。
Lumap/Services/LumapStore.swift	当前 UI 状态和学习用例编排、推荐、进度、奖励与 Persona。
Lumap/Services/LumapAIClient.swift	provider-neutral generation request、endpoint 解析、认证头和响应提取。
Lumap/Services/NarratedDeckService.swift	Deck 合同、远程/本地生成、校验、Kokoro 引擎池和播放状态。
Lumap/Services/LumapKeychainStore.swift	当前供应商 API 密钥的 Keychain 读写边界。
Lumap/Services/KokoroVoicePackManager.swift	语音包固定目录、导入 staging、安全检查与验证。
Lumap/Services/SpatialLearningCapability.swift	共享空间能力与 scenario 合同。
LumapiOS/Services/MobileARKitProvider.swift	iOS/Simulator 的 ARKit capability 判断。
Lumap/Views/LearningMethodModuleView.swift	macOS 方法 renderer，包括 Deck 和空间演示入口。
LumapiOS/Views/MobileNarratedDeckView.swift	iOS Deck、旁白、Quiz 和语音包状态交互。
LumapiOS/Views/MobileSpatialLearningView.swift	iOS 空间效果和 2D fallback。

B 图例与编译说明

本文不依赖网络图片；六张架构图全部由 TikZ 在编译时生成。仓库脚本使用 Tectonic 的 XeTeX 引擎，并自动重复编译以解析目录和交叉引用：

```
brew install tectonic
cd Lumap
./Scripts/compile-architecture-pdfs.sh
```

字体使用 macOS 自带的 Songti SC、Heiti SC、Times New Roman、Arial 与 Menlo。若在非 macOS 环境编译，应把字体声明替换为已安装的等价 CJK 与拉丁字体；公式和 TikZ 图本身不依赖外部资源。